

TRABALLO FIN DE GRAO
GRAO EN ENXEÑARÍA INFORMÁTICA
MENCIÓN EN ENXEÑARÍA DO SOFTWARE



PadelCenter: Aplicación web para la gestión de reservas de pistas de pádel y torneos

Estudiante: Breixo Camiña Fernández

Dirección: Javier Parapar López

Gilberto Pérez Vega

A Coruña, junio de 2026.

«Eres un carota, sinvergüenza, maleducado, palanquín e habilidoso.»

— Abuela Teresita

A mi padre, de quien heredé la lógica que me trajo hasta aquí.

A mi madre, que nunca bajó los brazos cuando yo estuve a punto de hacerlo.

A mi hermano, que no pudo disfrutar de todo lo que yo pude gracias al sacrificio de nuestros
padres.

A mi abuela Teresa, que me describió mejor de lo que yo jamás podría describirme a mí
mismo.

A mi abuela Nenucha, que ya no está para verlo, pero que siempre estuvo.

A Javi y Gilberto, que esperaron esta cena con una paciencia que no merezco.

A Ángel, que me enseñó que programar es lo menos importante de ser ingeniero informático.

A Paula, sin quien esto nunca hubiera ocurrido.

Agradecimientos

Quiero expresar mi agradecimiento, en primer lugar, a mis tutores Javier Parapar y Gilberto Silva, por su disposición y paciencia a lo largo de todos estos años. Su guía ha sido imprescindible para llegar hasta aquí.

A mis padres y hermano, por el esfuerzo y el sacrificio que hay detrás de este título.

A Paula, por ser el motor que puso en marcha lo que parecía no tener fin, y por aguantar con una generosidad infinita todo lo que conlleva compartir la vida con alguien que tiene un TFG pendiente.

A Esteban, mi amigo de toda la vida, por seguir ahí sin importar cuánto tiempo pasara entre una conversación y la siguiente.

A mis amigos de la infancia y a mis compañeros de universidad, por los momentos compartidos que hicieron más llevadero el camino.

Resumen

La digitalización de los clubes deportivos es una necesidad creciente en el contexto del auge del pádel en España. Sin embargo, la mayoría de los centros deportivos siguen gestionando sus reservas de forma manual o con herramientas genéricas que no cubren sus necesidades específicas. PadelCenter es una aplicación web diseñada para resolver este problema, ofreciendo una plataforma integral para la gestión de reservas de pistas de pádel y la organización de torneos.

El sistema se ha construido siguiendo principios de arquitectura hexagonal, separando el dominio de negocio de los detalles tecnológicos, lo que facilita el mantenimiento y la evolución del sistema. El backend está desarrollado en Java 21 con Spring Boot 3, empleando un enfoque *contract-first* basado en OpenAPI 3 para definir el contrato de la API antes de implementar cualquier lógica. El frontend es una aplicación Next.js 14 con App Router, TypeScript y React Query, con clientes HTTP generados automáticamente a partir de la especificación OpenAPI mediante Orval.

Las funcionalidades principales incluyen autenticación segura mediante JWT, la gestión de centros deportivos y pistas, la creación y consulta de reservas con control de conflictos, y la organización de torneos con generación automática de emparejamientos. La estrategia de pruebas abarca tests unitarios con Mockito, tests de integración con Testcontainers y tests de arquitectura con ArchUnit.

El resultado es un sistema funcional, bien estructurado y extensible, que demuestra la aplicabilidad de las prácticas modernas de ingeniería del software en el ámbito de la gestión deportiva.

Palabras clave:

- Arquitectura hexagonal
- Spring Boot
- Next.js
- API REST
- OpenAPI
- Gestión deportiva
- Reservas
- Torneos
- Java
- TypeScript

Índice general

1	Introducción	1
1.1	Contexto y motivación	1
1.2	Objetivos	2
1.2.1	Objetivos funcionales	2
1.2.2	Objetivos técnicos	2
1.3	Estructura de la memoria	2
2	Estado del arte	4
2.1	Soluciones existentes	4
2.1.1	Playtomic	4
2.1.2	Smashapp	4
2.1.3	Sistemas propios de clubes	5
2.1.4	Análisis comparativo	5
2.2	Tecnologías evaluadas	5
2.2.1	Frameworks de backend	5
2.2.2	Frameworks de frontend	6
2.3	Arquitectura hexagonal vs. arquitectura en capas	6
2.3.1	Ports & Adapters: origen y conceptos	7
2.3.2	Un ejemplo concreto: BookingRepository	8
3	Metodología y planificación	10
3.1	Metodología de desarrollo	10
3.1.1	Herramientas de seguimiento	11
3.2	Plan de trabajo inicial	11
3.2.1	Análisis de riesgos	11
3.3	Ejecución real y desviaciones	11

4	Análisis	15
4.1	Actores del sistema	15
4.2	Requisitos funcionales	16
4.2.1	Autenticación y usuarios	16
4.2.2	Gestión de centros	16
4.2.3	Gestión de pistas	16
4.2.4	Gestión de reservas	17
4.2.5	Gestión de torneos	17
4.3	Ciclo de vida de un torneo	18
4.4	Requisitos no funcionales	19
4.5	Casos de uso	20
4.5.1	Caso de uso detallado: Crear una reserva (CU-Booking-Create)	20
4.5.2	Caso de uso detallado: Cerrar inscripción y generar cuadro (CU-Tournament-CloseBracket)	20
5	Diseño	23
5.1	Arquitectura hexagonal	23
5.1.1	Modelos de dominio como records inmutables	23
5.2	Modelo de dominio	24
5.3	Diseño de la API — Contract-First	25
5.3.1	Convenciones de la especificación	25
5.3.2	Endpoints de la API	26
5.4	Esquema de base de datos	26
5.5	Diseño de seguridad	27
5.5.1	Evaluadores de autorización personalizados	27
6	Implementación	29
6.1	Backend	29
6.1.1	Estructura del proyecto	29
6.1.2	Generación de código OpenAPI	30
6.1.3	Patrón de mapeo en dos capas	30
6.1.4	Flyway y migraciones de base de datos	31
6.1.5	Virtual threads (Project Loom)	31
6.1.6	Trazabilidad con MDC y logging estructurado	32
6.1.7	Convenciones de los casos de uso	32
6.2	Frontend	32
6.2.1	Estructura del proyecto	32
6.2.2	Librería de componentes: shadcn/ui y Radix UI	32

6.2.3	Generación de hooks con Orval	33
6.2.4	Gestión del estado de autenticación con Zustand	33
6.2.5	Protección de rutas	33
6.3	Flujo end-to-end: creación de una reserva	33
6.4	Entorno local con Docker Compose	34
6.4.1	Colección Postman	34
7	Pruebas	35
7.1	Estrategia de pruebas	35
7.2	Tests unitarios con Mockito	36
7.3	Tests de integración con Testcontainers	36
7.4	Tests de arquitectura con ArchUnit	37
7.5	Resumen de cobertura	37
8	Conclusiones	39
8.1	Objetivos alcanzados	39
8.2	Dificultades encontradas	40
8.3	Relación con las competencias de la titulación	40
8.4	Consideraciones éticas, legales y de sostenibilidad	41
8.4.1	Protección de datos personales	41
8.4.2	Impacto social y económico	41
8.4.3	Sostenibilidad	42
8.4.4	Viabilidad económica	42
8.5	Líneas de trabajo futuro	42
A	Material adicional	45
A.1	Migración V4 — Índice de solapamiento de reservas	45
A.2	Algoritmo de generación de cuadros de torneo	46

Índice de figuras

4.1	Ciclo de vida de un torneo: estados, descripción y transiciones.	18
5.1	Estructura de capas de la arquitectura hexagonal de PadelCenter.	24

Índice de tablas

2.1	Comparativa de soluciones existentes.	5
3.1	Plan de trabajo inicial por fases, con esfuerzo estimado como porcentaje del total.	12
3.2	Riesgos identificados al inicio del proyecto y mitigaciones aplicadas.	13
3.3	Actividad real de desarrollo por periodo, según el historial de control de versiones.	14
4.1	Casos de uso del sistema agrupados por dominio.	21
5.1	Endpoints de la API REST de PadelCenter.	28
7.1	Resumen de tests y cobertura del backend (medida con JaCoCo, perfil mvn <code>clean verify</code>).	37

Introducción

EL pádel es el segundo deporte más practicado en España, con más de cinco millones de jugadores federados y miles de clubes repartidos por todo el territorio nacional. Este crecimiento exponencial ha generado una demanda creciente de herramientas digitales que permitan a los centros deportivos gestionar su actividad de forma eficiente, desde la reserva de pistas hasta la organización de torneos. Sin embargo, muchos clubes siguen dependiendo de sistemas manuales, hojas de cálculo o aplicaciones genéricas que no están adaptadas a las particularidades del negocio del pádel.

PadelCenter nace como respuesta a esta necesidad: una aplicación web específicamente diseñada para la gestión integral de centros de pádel, que abarca la administración de pistas, la creación y gestión de reservas, y la organización de torneos con emparejamientos automáticos.

1.1 Contexto y motivación

La transformación digital del sector deportivo es una tendencia global impulsada por la pandemia de COVID-19 y por el auge de los servicios *on-demand*. Los usuarios esperan poder reservar una pista desde su teléfono móvil en cuestión de segundos, con confirmación inmediata y sin necesidad de llamar por teléfono al club.

Desde el punto de vista académico, este proyecto representa una oportunidad de aplicar en un contexto real las competencias adquiridas durante el Grado en Ingeniería Informática, en particular las relacionadas con la ingeniería del software: diseño de sistemas, arquitecturas limpias, desarrollo *contract-first*, pruebas automatizadas y trabajo con tecnologías modernas tanto en backend como en frontend.

La elección del dominio del pádel responde a una motivación personal del autor, combinada con la observación de que los sistemas existentes en el mercado presentan importantes carencias en cuanto a personalización, extensibilidad y calidad del software subyacente.

1.2 Objetivos

Los objetivos del presente trabajo se dividen en funcionales y técnicos:

1.2.1 Objetivos funcionales

- Implementar un sistema de autenticación seguro con roles diferenciados (jugador y administrador de centro).
- Permitir la gestión de centros deportivos y sus pistas (alta, baja, modificación, consulta).
- Desarrollar un módulo de reservas con control de conflictos temporales y visibilidad según el rol del usuario.
- Implementar un módulo de torneos con inscripción de participantes y generación automática de emparejamientos.

1.2.2 Objetivos técnicos

- Diseñar el sistema siguiendo la arquitectura hexagonal, garantizando la independencia del dominio de negocio respecto a los detalles de infraestructura.
- Adoptar un enfoque *contract-first* basado en OpenAPI 3, de modo que la especificación de la API sea la fuente de verdad del contrato entre frontend y backend.
- Garantizar una cobertura de pruebas adecuada mediante tests unitarios, de integración y de arquitectura.
- Implementar un entorno de desarrollo reproducible mediante Docker Compose.

1.3 Estructura de la memoria

El presente documento se organiza en los siguientes capítulos:

Capítulo 2 — Estado del arte: Se analizan las soluciones existentes en el mercado para la gestión de reservas deportivas, se evalúan las tecnologías candidatas y se justifica la elección de la pila tecnológica adoptada.

Capítulo 3 — Metodología y planificación: Se describe la metodología de desarrollo adoptada, el plan de trabajo inicial, los riesgos identificados y un análisis de la ejecución real del proyecto frente a lo planificado.

Capítulo 4 — Análisis: Se presentan los actores del sistema, los requisitos funcionales y no funcionales, y los casos de uso principales.

Capítulo 5 — Diseño: Se describe la arquitectura del sistema, el modelo de dominio, el diseño de la API y el esquema de base de datos.

Capítulo 6 — Implementación: Se detallan los aspectos más relevantes de la implementación del backend y el frontend, así como la integración entre ambos.

Capítulo 7 — Pruebas: Se explica la estrategia de pruebas adoptada y se presentan los resultados obtenidos.

Capítulo 8 — Conclusiones: Se evalúan los resultados obtenidos, se relaciona el trabajo con las competencias de la titulación y se proponen líneas de trabajo futuro.

Estado del arte

LA gestión digital de instalaciones deportivas es un mercado en rápido crecimiento, impulsado por la popularización del pádel y la demanda de servicios *on-demand*. En este capítulo se analizan las soluciones existentes, se evalúan las tecnologías candidatas para el desarrollo del sistema y se justifica la elección tecnológica adoptada.

2.1 Soluciones existentes

2.1.1 Playtomic

Playtomic es la plataforma líder en España para la reserva de pistas de pádel y tenis. Permite a los usuarios buscar pistas disponibles en su zona, realizar reservas en tiempo real y gestionar sus partidos. Desde el punto de vista del club, ofrece un panel de administración para gestionar disponibilidad, precios y bonos.

Sus principales fortalezas son la amplia red de clubes integrados y la experiencia de usuario en la aplicación móvil. Sin embargo, presenta limitaciones relevantes: es una solución SaaS cerrada sin posibilidad de personalización, cobra una comisión por reserva que puede resultar elevada para clubes pequeños, y no ofrece funcionalidades avanzadas de gestión de torneos integradas con el sistema de reservas.

2.1.2 Smashapp

Smashapp es una plataforma orientada a la gestión de ligas y torneos de pádel. Su punto fuerte es el módulo de competición, que permite crear torneos, gestionar inscripciones y publicar resultados. No obstante, sus capacidades de gestión de reservas son limitadas y está orientado a competiciones federadas más que a la gestión cotidiana de un club.

2.1.3 Sistemas propios de clubes

Muchos clubes han desarrollado o contratado sistemas a medida, a menudo implementados como simples formularios web conectados a una base de datos o incluso como hojas de cálculo compartidas. Estos sistemas presentan problemas de mantenibilidad, falta de integración entre módulos y dependencia de un proveedor concreto.

2.1.4 Análisis comparativo

La tabla 2.1 resume las funcionalidades principales de las soluciones analizadas. Playtomic y Smashapp son productos cerrados sin documentación técnica pública ni código accesible, por lo que la comparación se basa en el uso de sus aplicaciones y en su documentación de cara al cliente, no en una auditoría de su implementación; debe leerse como una valoración cualitativa orientativa —suficiente para justificar la elección de funcionalidades de PadelCenter— y no como una verificación exhaustiva de las capacidades reales de cada competidor.

Funcionalidad	Playtomic	Smashapp	Sist. propios	PadelCenter
Reserva de pistas	Sí	Limitado	Variable	Sí
Gestión de torneos	Básico	Sí	Raro	Sí
Emparejamientos auto.	No	Sí	No	Sí
Personalizable	No	Limitado	Sí	Sí
Código abierto	No	No	Variable	Sí
Sin comisión por reserva	No	No	Sí	Sí
Arquitectura moderna	Descono.	Descono.	No	Sí

Tabla 2.1: Comparativa de soluciones existentes.

2.2 Tecnologías evaluadas

2.2.1 Frameworks de backend

Se evaluaron tres frameworks Java para el backend:

Spring Boot [?] Es el framework más maduro y ampliamente adoptado del ecosistema Java. Ofrece un ecosistema de módulos muy completo (Spring Security, Spring Data JPA, Spring MVC), una comunidad enorme y excelente integración con herramientas de generación de código a partir de OpenAPI. Su modelo de programación es familiar para cualquier desarrollador Java.

Quarkus Framework diseñado para entornos cloud-native, con tiempos de arranque muy reducidos y menor huella de memoria. Aunque es una opción muy interesante para microservicios, su ecosistema es menos maduro que Spring Boot y la curva de aprendizaje es mayor para quienes provienen del mundo Java EE.

Micronaut Similar a Quarkus en su orientación cloud-native, con inyección de dependencias en tiempo de compilación. Sus ventajas en rendimiento son similares a las de Quarkus, pero su adopción en el mercado es significativamente menor, lo que implica menor disponibilidad de recursos y librerías.

La elección recayó sobre **Spring Boot** por su madurez, la calidad del plugin OpenAPI Generator para Maven y la cobertura de todas las necesidades del proyecto sin compromisos.

2.2.2 Frameworks de frontend

Se evaluaron tres frameworks JavaScript/TypeScript para el frontend:

Next.js [?] Framework React con renderizado híbrido (SSR, SSG y RSC), App Router en su versión 14, y un ecosistema muy activo. Su modelo de enrutamiento basado en el sistema de ficheros es intuitivo y facilita la organización del código. La integración con React Query y la generación de código con Orval es directa.

Nuxt Equivalente de Next.js para el ecosistema Vue.js. Ofrece características similares pero la comunidad de Vue es más pequeña y el autor tiene mayor experiencia con React.

Angular Framework completo de Google, con un enfoque más opinado y una mayor cantidad de boilerplate. Su curva de aprendizaje es más pronunciada y su arquitectura, aunque sólida, resulta excesiva para un proyecto de este tamaño.

La elección recayó sobre **Next.js** por su flexibilidad, la calidad del ecosistema React y la compatibilidad con las herramientas de generación de código elegidas.

2.3 Arquitectura hexagonal vs. arquitectura en capas

La arquitectura en capas tradicional (*Presentation* $\rightarrow$ *Service* $\rightarrow$ *Repository*) es el patrón más extendido en aplicaciones Spring Boot. Sin embargo, presenta un problema fundamental: las capas superiores acaban dependiendo de detalles de infraestructura (JPA, SQL, HTTP) que se filtran a través de las capas, acoplando el dominio de negocio a tecnologías concretas. En la práctica, el *Service* suele inyectar directamente una interfaz *JpaRepository* de Spring Data: el dominio de negocio queda comprometido, en tiempo de compilación, con un *framework* de persistencia concreto.

2.3.1 Ports & Adapters: origen y conceptos

La arquitectura hexagonal, también conocida como *Ports & Adapters*, fue propuesta por Alistair Cockburn [?] para resolver precisamente ese problema, partiendo de una observación: tanto la interfaz de usuario como la base de datos son, desde el punto de vista del núcleo de la aplicación, igualmente *externas*. La forma hexagonal no tiene un significado literal de seis lados; representa simplemente que el núcleo puede exponer tantos lados (puertos) como necesite, a diferencia del rectángulo de arriba a abajo de la arquitectura en capas, que sugiere una única dirección de dependencia lineal.

Cockburn distingue dos tipos de puerto:

Puertos primarios (*driving*): interfaces que el núcleo *ofrece* hacia el exterior. Un caso de uso (`CreateBookingUseCase`) es un puerto primario: algo de fuera —un controlador REST, una prueba automatizada, una futura interfaz de línea de comandos— lo invoca para que el núcleo haga su trabajo.

Puertos secundarios (*driven*): interfaces que el núcleo *necesita* del exterior para funcionar. `BookingRepository` es un puerto secundario: el núcleo lo invoca, pero quien lo *implementa* está fuera del núcleo.

Cada puerto tiene uno o varios *adaptadores* que lo conectan con una tecnología concreta. Los adaptadores primarios *dirigen* el núcleo (un controlador Spring MVC que traduce una petición HTTP en una llamada a `CreateBookingUseCase.execute()`); los adaptadores secundarios son *dirigidos por* el núcleo (`BookingRepositoryImpl`, que traduce las llamadas del puerto a consultas JPA). La idea distintiva del patrón —más allá de la simple inyección de dependencias— es esta **simetría**: desde la perspectiva del núcleo da igual si quien lo invoca es un controlador REST, una cola de mensajes o un test unitario (son adaptadores primarios intercambiables), y da igual si la persistencia es PostgreSQL, otra base de datos o un doble de prueba en memoria (son adaptadores secundarios intercambiables). El núcleo solo conoce las interfaces; nunca conoce qué adaptador concreto está al otro lado.

Esta misma idea de inversión de dependencias hacia el núcleo es el principio central de la *Clean Architecture* de Robert C. Martin [?], que reformula los dos tipos de puerto de Cockburn como anillos concéntricos (entidades, casos de uso, adaptadores de interfaz, *frameworks* y controladores) gobernados por la *Dependency Rule*: las dependencias de código solo pueden apuntar hacia el interior, nunca hacia afuera. PadelCenter sigue esta idea de forma directa, organizando el código en tres paquetes que se corresponden con los conceptos de Cockburn: `domain` y `application` forman el núcleo hexagonal (entidades y casos de uso, que definen los puertos como interfaces Java); `infrastructure.inbound` contiene los adaptadores primarios (controladores REST); e `infrastructure.outbound` contiene los adaptadores secundarios (repositorios JPA, el adaptador de seguridad JWT). El

propio patrón de mapeo en dos capas descrito en la sección 6.1 recoge ideas clásicas de mapeo objeto-relacional descritas por Fowler [?], adaptadas para mantener las entidades JPA completamente fuera del dominio.

2.3.2 Un ejemplo concreto: **BookingRepository**

El puerto secundario `BookingRepository` (interfaz en `domain.repository`) declara, entre otras, la operación que detecta conflictos de horario:

```
1 public interface BookingRepository {  
2     boolean existsOverlappingBooking(  
3         UUID fieldId, LocalDateTime start,  
4         LocalDateTime end, Long excludeId);  
5     // ...  
6 }
```

`CreateBookingUseCase` depende únicamente de esta interfaz: ni la importa, ni puede importar, ningún tipo de Spring Data JPA. El adaptador secundario que la implementa, `BookingRepositoryImpl`, vive en `infrastructure.outbound` y es el único punto del sistema que conoce a la vez el puerto del dominio y la API de Spring Data JPA:

```
1 @Repository  
2 public class BookingRepositoryImpl implements BookingRepository {  
3     private final BookingJpaRepository bookingJpaRepository;  
4     private final BookingPersistenceMapper mapper;  
5     // delega en bookingJpaRepository y traduce  
6     // entidades JPA <-> records del dominio  
7 }
```

En una arquitectura en capas tradicional, `CreateBookingUseCase` (o el equivalente `BookingService`) inyectaría directamente `BookingJpaRepository`, una interfaz de Spring Data: el dominio importaría `org.springframework.data.jpa.repository.JpaRepository` y quedaría acoplado a JPA en tiempo de compilación, exactamente el problema señalado al inicio de esta sección. Con el puerto de por medio, esa dependencia desaparece del dominio: Spring conecta caso de uso y adaptador en tiempo de *ejecución* mediante inyección de dependencias, pero en tiempo de *compilación* `application` no depende de `infrastructure` en ningún sentido —la regla que ArchUnit verifica automáticamente en cada build (sección 7.4).

Las ventajas concretas para PadelCenter, consecuencia directa de esta inversión, son:

- Posibilidad de testar los casos de uso sin levantar el contexto Spring: un test sustituye `BookingRepository` por un doble de prueba (Mockito), sin que el caso de uso note la diferencia.

- Facilidad para sustituir la base de datos o el mecanismo de autenticación sin modificar el dominio: bastaría con escribir un nuevo adaptador que implemente el mismo puerto.
- Reglas de arquitectura verificables automáticamente con ArchUnit.

La desventaja principal es el mayor número de clases e interfaces necesarias respecto a una arquitectura en capas plana. Sin embargo, en un proyecto con ambición de mantenibilidad a largo plazo, esta inversión es justificada.

Metodología y planificación

EL desarrollo de PadelCenter se ha llevado a cabo de forma individual, sin un equipo que requiera ceremonias de coordinación, pero sí con una metodología de trabajo deliberada que ha guiado el orden de las tareas, la gestión del riesgo y el seguimiento del progreso. En este capítulo se describe dicha metodología, la planificación inicial del trabajo y un análisis de cómo la ejecución real se desvió de lo previsto.

3.1 Metodología de desarrollo

Se adoptó un enfoque **iterativo e incremental por módulo funcional**, combinado con el carácter *contract-first* ya descrito en el capítulo 5. Cada módulo (autenticación, centros y pistas, reservas, torneos) se desarrolló siguiendo el mismo ciclo:

1. Definición del contrato del módulo en la especificación OpenAPI.
2. Diseño del modelo de dominio y los puertos correspondientes.
3. Implementación de los casos de uso y los adaptadores de infraestructura.
4. Cobertura con tests unitarios y de integración.
5. Implementación del frontend que consume el módulo.

Este ciclo se eligió frente a un desarrollo en cascada porque permite obtener, al final de cada iteración, un incremento funcional completo y verificable (*vertical slice*), reduciendo el riesgo de descubrir problemas de integración entre capas al final del proyecto. Frente a un marco más formal como Scrum, se descartaron las ceremonias de equipo (*daily*, *sprint planning* con varios roles, retrospectivas grupales) por no aportar valor en un trabajo individual, conservando en cambio los elementos que sí son útiles en solitario: tablero de tareas, iteraciones acotadas en el tiempo y revisión del progreso al cierre de cada una.

3.1.1 Herramientas de seguimiento

- **Control de versiones y tablero de tareas:** Git, con un repositorio independiente para backend y frontend, e issues para registrar tareas pendientes y errores detectados durante el desarrollo.
- **Integración continua:** ejecución automática de la batería de tests (unitarios, integración y arquitectura) en cada cambio, evitando que se acumule deuda técnica no detectada.
- **Especificación OpenAPI:** además de contrato de API, ha actuado como documento vivo de seguimiento del alcance funcional: el progreso de cada módulo es directamente visible en el número de endpoints especificados e implementados.

3.2 Plan de trabajo inicial

La planificación inicial dividió el trabajo en cinco fases, alineadas con la estructura de la memoria:

3.2.1 Análisis de riesgos

3.3 Ejecución real y desviaciones

La figura 3.3 resume la actividad real de desarrollo por mes, medida a partir del historial de control de versiones de ambos repositorios.

La desviación más relevante respecto al plan inicial es la **concentración del trabajo en un periodo mucho más corto del previsto**: la fase de análisis y diseño (F1) se extendió de forma discontinua durante varios meses por solapamiento con las obligaciones laborales del autor, mientras que las fases F2 y F3 (backend y frontend), planificadas inicialmente de forma secuencial, terminaron ejecutándose de manera prácticamente simultánea durante abril de 2026 una vez estabilizado el contrato de la API. Esto fue posible precisamente gracias al enfoque *contract-first*: al existir un contrato OpenAPI estable, ambos proyectos pudieron avanzar en paralelo sin bloquearse mutuamente, ya que el frontend podía generar sus tipos y *hooks* a partir de la especificación sin depender de que el backend estuviese completamente implementado.

En términos de esfuerzo relativo, la proporción final entre backend y frontend (78 *commits* frente a 20) es más desequilibrada que la estimada inicialmente en la tabla 3.1 (40 % frente a 25 %). Esto se explica por el mayor número de módulos de dominio a modelar en el backend (siete entidades principales, 42 casos de uso) frente a un frontend que reutiliza en gran medida

los mismos patrones de página para cada módulo una vez establecida la estructura inicial de rutas y componentes.

Fase	Contenido	Esfuerzo estimado
F1 — Análisis y diseño	Estado del arte, definición de requisitos, diseño de arquitectura y modelo de dominio, primera versión del contrato OpenAPI	20%
F2 — Backend	Implementación de dominio, casos de uso, persistencia, seguridad y tests del backend	40%
F3 — Frontend	Implementación de la interfaz, integración con la API generada, gestión de estado y autenticación en cliente	25%
F4 — Pruebas e integración	Tests de integración end-to-end, pulido de errores de integración entre frontend y backend	10%
F5 — Documentación	Redacción de la memoria y preparación de la defensa	5%

Tabla 3.1: Plan de trabajo inicial por fases, con esfuerzo estimado como porcentaje del total.

Riesgo

Disponibilidad irregular por solapamiento con la actividad laboral del autor

Cambios en el contrato de la API tras iniciar la implementación del frontend

Periodo	Backend	Frontend
Noviembre 2025 – febrero 2026	Actividad baja y discontinua: diseño de la arquitectura hexagonal, modelo de dominio inicial y primera versión del contrato OpenAPI, en paralelo con la actividad laboral del autor	Sin actividad: el frontend no se inició hasta tener estabilizado el contrato de la API
Marzo 2026	Sin actividad registrada	Sin actividad registrada
Abril 2026	Fase intensiva: implementación de la mayor parte de los casos de uso, persistencia, seguridad y tests	Fase intensiva: implementación de la mayor parte de las páginas, integración con la API generada y gestión de estado
Mayo – junio 2026	Cierre, correcciones puntuales y redacción de la memoria	Cierre, correcciones puntuales y redacción de la memoria

Tabla 3.3: Actividad real de desarrollo por periodo, según el historial de control de versiones.

Capítulo 4

Análisis

EL análisis del sistema parte de la identificación de los actores que interaccionan con Pa-delCenter y de las necesidades que cada uno de ellos tiene. A partir de estos actores se derivan los requisitos funcionales y no funcionales, que constituyen la base del diseño y la implementación.

4.1 Actores del sistema

Se han identificado tres actores con responsabilidades diferenciadas, modelados mediante el sistema de roles por centro (`CenterRole`):

Usuario (USER): Jugador registrado en el sistema. Puede consultar centros y pistas disponibles, crear y cancelar sus propias reservas, inscribirse en torneos como pareja y consultar sus partidos. Tiene visibilidad limitada: solo accede a sus propios datos.

Manager (MANAGER): Gestor operativo de un centro concreto. Puede consultar todas las reservas del centro que gestiona, gestionar la disponibilidad de pistas y administrar los torneos del centro. Su ámbito de actuación está restringido al centro al que está asignado.

Administrador (ADMIN): Administrador global del sistema. Tiene acceso completo: puede crear y eliminar centros, asignar roles a usuarios, gestionar cualquier entidad del sistema y acceder a todos los datos independientemente del centro.

El rol es contextual por centro: un mismo usuario puede ser ADMIN en un centro y USER en otro. Esto se modela mediante la entidad `CenterRole`, que asocia un usuario con un rol para un centro concreto.

4.2 Requisitos funcionales

4.2.1 Autenticación y usuarios

- **RF-01:** El sistema permite el registro de nuevos usuarios con nombre, apellidos, correo electrónico, contraseña y teléfono.
- **RF-02:** El sistema permite el inicio de sesión mediante credenciales (correo y contraseña), devolviendo un *access token* JWT.
- **RF-03:** El *access token* expira a los 15 minutos; el cliente puede renovarlo mediante un *refresh token* almacenado en cookie *httpOnly*.
- **RF-04:** Cada usuario tiene un rol global y opcionalmente uno o varios roles por centro (*CenterRole*).
- **RF-05:** Un administrador puede modificar los roles de cualquier usuario y cambiar su contraseña.

4.2.2 Gestión de centros

- **RF-06:** Un administrador puede crear un centro deportivo con nombre, dirección, ciudad, teléfono y correo de contacto.
- **RF-07:** Un administrador puede eliminar un centro (soft delete), siempre que no tenga pistas activas asociadas; en caso contrario la operación se rechaza con un conflicto.
- **RF-08:** Cualquier usuario autenticado puede consultar el listado paginado de centros activos y el detalle de un centro.

4.2.3 Gestión de pistas

- **RF-09:** Un administrador puede dar de alta pistas en sus centros, indicando nombre, tipo de superficie, precio por hora y disponibilidad.
- **RF-10:** Un administrador puede activar o desactivar una pista (*isAvailable*) sin eliminarla.
- **RF-11:** Cualquier usuario autenticado puede consultar la disponibilidad de una pista para una fecha concreta.
- **RF-12:** Un administrador puede eliminar una pista.

4.2.4 Gestión de reservas

- **RF-13:** Un usuario puede crear una reserva para una pista en una franja horaria concreta.
- **RF-14:** El sistema rechaza la reserva si la pista no está disponible o si ya existe otra reserva activa en el mismo tramo horario (control de conflictos mediante índice de solapamiento en base de datos).
- **RF-15:** Una reserva pasa por los estados: PENDING → CONFIRMED → COMPLETED/CANCELLED.
- **RF-16:** Un usuario puede cancelar sus propias reservas; un administrador o manager puede cancelar cualquier reserva de su centro.
- **RF-17:** Un usuario puede consultar sus reservas actuales e históricas (GET /api/v1/me/bookings).
- **RF-18:** Un administrador o manager puede consultar todas las reservas de un centro (GET /api/v1/centers/{id}/bookings).

4.2.5 Gestión de torneos

- **RF-19:** Un administrador puede crear un torneo en uno de sus centros con nombre, descripción, formato de competición y número máximo de parejas.
- **RF-20:** Los formatos de competición soportados son: ROUND_ROBIN (todos contra todos), ELIMINATION (eliminación directa) y GROUPS_AND_ELIMINATION (fase de grupos más eliminación).
- **RF-21:** Un administrador puede abrir y cerrar el período de inscripción del torneo.
- **RF-22:** Los jugadores se inscriben en el torneo formando parejas. La pareja pasa por los estados PENDING → CONFIRMED/REJECTED.
- **RF-23:** Al cerrar la inscripción, el sistema genera automáticamente los emparejamientos del cuadro según el formato elegido.
- **RF-24:** Un administrador puede registrar el resultado de cada partido (scoreA, scoreB, ganador).
- **RF-25:** El sistema calcula y expone la clasificación en tiempo real para torneos en formato Round Robin.

- **RF-26:** Un usuario puede consultar sus torneos (GET /api/v1/me/tournaments) y sus partidos (GET /api/v1/me/matches).

4.3 Ciclo de vida de un torneo

El torneo sigue un ciclo de vida estricto con seis estados:

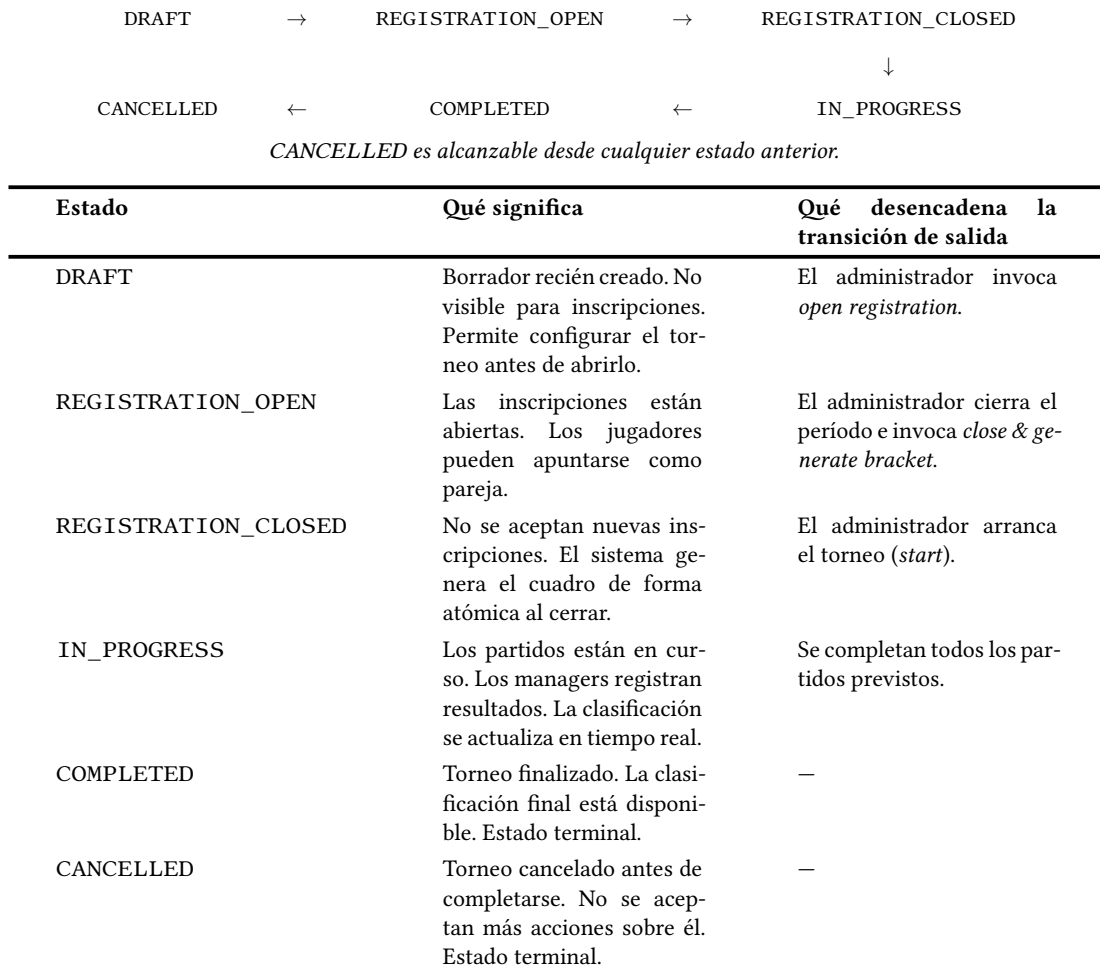


Figura 4.1: Ciclo de vida de un torneo: estados, descripción y transiciones.

La transición *close & generate bracket* es atómica: cierra la inscripción y genera todos los emparejamientos iniciales en una misma operación (*CloseRegistrationAndGenerateBracketUseCase*), que delega en *BracketGeneratorService* (servicio de dominio puro, sin dependencias de persistencia) la elección del algoritmo según el formato:

ROUND_ROBIN: todos contra todos. Se genera un partido por cada pareja de parejas dis-

tinta —equivalente a las tablas de *round-robin scheduling* de Berger, habituales en la programación de ligas—, sin restricción sobre el número de parejas inscritas.

ELIMINATION: eliminación directa. Las parejas se mezclan aleatoriamente y se emparejan para la primera ronda; las rondas siguientes se crean como partidos *placeholder* sin parejas asignadas, que se completan progresivamente a medida que se conocen los ganadores (detalle completo en el anexo A.2). Este formato exige que el número de parejas confirmadas sea potencia de 2.

GROUPS_AND_ELIMINATION: fase de grupos de 4 parejas con *round robin* interno, seguida de una fase eliminatoria entre los dos primeros clasificados de cada grupo.

Se descartó adoptar una librería de generación de cuadros de torneo existente porque ninguna de las candidatas evaluadas soporta de forma nativa los tres formatos sobre los tipos de dominio propios de PadelCenter (`TournamentPair`, `Match`); habría sido necesaria una capa de adaptación entre la librería y el dominio cuya complejidad habría superado la del propio algoritmo, que es deliberadamente simple y no tiene dependencias externas.

4.4 Requisitos no funcionales

RNF-01 — Seguridad: Toda la API requiere autenticación mediante JWT Bearer token, salvo los endpoints de registro, login, refresh y logout. Las contraseñas se almacenan con BCrypt. El *refresh token* se transmite exclusivamente mediante cookie *httpOnly* y *SameSite=Strict* para mitigar ataques XSS y CSRF.

RNF-02 — Autorización granular: La autorización se gestiona mediante evaluadores `custom` (`@bookingOwnerEvaluator`, `@centerRoleEvaluator`, `@tournamentRoleEvaluator`) que se invocan desde `@PreAuthorize` en los casos de uso, garantizando que cada operación solo es accesible por el actor con el rol correcto en el contexto adecuado.

RNF-03 — Rendimiento: El backend utiliza virtual threads de Java 21 (*Project Loom*), activados mediante `spring.threads.virtual.enabled=true`. Esto permite manejar un número mucho mayor de conexiones concurrentes sin saturar el pool de hilos del sistema operativo.

RNF-04 — Contrato API: La especificación OpenAPI es la fuente de verdad. Cualquier cambio en la API debe reflejarse primero en el YAML y después propagarse al código generado tanto en backend como en frontend.

RNF-05 — Reproducibilidad: El entorno de desarrollo es completamente reproducible mediante Docker Compose. Cualquier desarrollador puede levantar el stack completo con un único comando.

RNF-06 — Mantenibilidad: La arquitectura hexagonal garantiza que el dominio de negocio no depende de ningún framework. Las reglas de arquitectura se verifican automáticamente en cada build mediante ArchUnit.

RNF-07 — Trazabilidad: Todos los logs se emiten en formato JSON estructurado (Logstash encoder) con contexto MDC que incluye el identificador de usuario y el ID de correlación de la petición HTTP.

4.5 Casos de uso

El sistema implementa 42 casos de uso organizados en cuatro categorías. La tabla 4.1 recoge los principales agrupados por dominio.

4.5.1 Caso de uso detallado: Crear una reserva (CU-Booking-Create)

Actor principal: Usuario autenticado con rol USER.

Precondición: El usuario ha iniciado sesión, la pista existe y está activa (`isAvailable = true`).

Flujo principal:

1. El usuario selecciona una pista e indica fecha, hora de inicio y hora de fin.
2. El sistema verifica que la pista está disponible (`isAvailable`).
3. El sistema consulta el índice de solapamiento en base de datos para detectar conflictos con reservas activas en el mismo tramo horario.
4. El sistema crea la reserva con estado `PENDING`, calcula el precio total y la persiste.
5. El sistema devuelve los datos de la reserva con HTTP 201.

Flujo alternativo — Conflicto de horario:

3. El sistema detecta solapamiento y devuelve HTTP 409 con detalle de error según RFC 9457.

Postcondición: La reserva queda registrada con estado `PENDING` y la pista queda bloqueada para ese tramo horario.

Dominio	Actor	Casos de uso
Auth / Usuario	Anónimo	Registrarse, iniciar sesión, renovar token, cerrar sesión
	Usuario	Ver perfil propio, actualizar contraseña
	Admin	Listar usuarios, cambiar roles, eliminar usuario
Centros	Admin	Crear centro, eliminar centro
	Todos	Listar centros, ver detalle de centro
Pistas	Admin/Manager	Crear pista, actualizar disponibilidad, eliminar pista
	Todos	Listar pistas, ver disponibilidad por fecha
Reservas	Usuario	Crear reserva, cancelar reserva propia, ver mis reservas
	Admin/Manager	Ver reservas del centro, cancelar cualquier reserva
	Todos	Ver detalle de reserva
Torneos	Admin	Crear torneo, abrir inscripción, cerrar inscripción y generar cuadro, eliminar torneo
	Admin/Manager	Confirmar pareja, reportar resultado de partido
	Usuario	Inscribirse como pareja, ver mis torneos, ver mis partidos
	Todos	Ver cuadro, ver partidos, ver clasificación (Round Robin)

Tabla 4.1: Casos de uso del sistema agrupados por dominio.

4.5.2 Caso de uso detallado: Cerrar inscripción y generar cuadro (CU-Tournament-CloseBracket)

Actor principal: Administrador con rol ADMIN o MANAGER en el centro del torneo.

Precondición: El torneo está en estado REGISTRATION_OPEN y tiene al menos dos parejas confirmadas.

Flujo principal:

1. El administrador invoca el endpoint de cierre de inscripción.
2. El sistema cambia el estado del torneo a REGISTRATION_CLOSED.

3. El sistema genera automáticamente los emparejamientos iniciales según el formato del torneo (Round Robin, Eliminación o Grupos).
4. Los partidos se crean con estado SCHEDULED.
5. El sistema devuelve el cuadro generado con HTTP 200.

Postcondición: El torneo tiene un cuadro completo y está listo para comenzar.

Capítulo 5

Diseño

EL diseño de PadelCenter gira en torno a tres decisiones arquitectónicas fundamentales: la arquitectura hexagonal como estructura del backend, el enfoque *contract-first* con OpenAPI como contrato de la API, y OAuth2 Resource Server con JWT local como mecanismo de seguridad. En este capítulo se describen estas decisiones y sus consecuencias en la estructura del sistema.

5.1 Arquitectura hexagonal

La arquitectura hexagonal organiza el backend en tres capas con dependencias estrictamente dirigidas hacia el interior:

domain: Contiene los modelos de negocio, los enumerados, los puertos (interfaces) y las excepciones de dominio. No tiene ninguna dependencia externa: ni Spring, ni JPA, ni ninguna otra librería de infraestructura. Es el núcleo del sistema.

application: Contiene los casos de uso, organizados en cuatro subpaquetes según su naturaleza: `create`, `read`, `update` y `delete`. También define los tipos `CommandUseCase` y `QueryUseCase` como interfaces base. Solo depende de `domain`.

infrastructure: Contiene los adaptadores de entrada (`inbound`: controladores HTTP, mappers de API) y de salida (`outbound`: entidades JPA, repositorios, mappers de persistencia, seguridad). Implementa los puertos definidos en `domain`.

5.1.1 Modelos de dominio como records inmutables

Una decisión de diseño relevante es que todos los modelos del dominio son **Java records** (introducidos en Java 16 y estables desde Java 17). Los records son clases inmutables por defi-

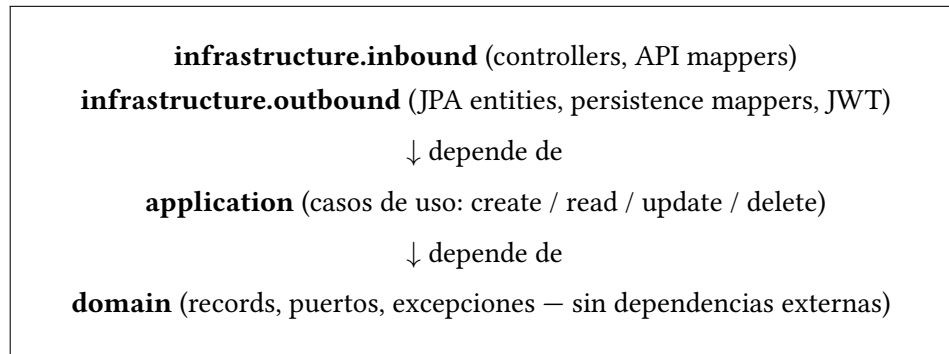


Figura 5.1: Estructura de capas de la arquitectura hexagonal de PadelCenter.

nición: todos sus campos se declaran en el constructor canónico, no tienen setters y proporcionan `equals`, `hashCode` y `toString` automáticamente.

Esta elección refuerza la arquitectura hexagonal de dos formas:

- Ningún adaptador puede modificar un objeto de dominio una vez creado, evitando efectos secundarios difíciles de rastrear.
- Los records no pueden extender otras clases, lo que impide que las entidades de dominio hereden accidentalmente de clases de infraestructura (como entidades JPA).

Las entidades JPA (`UserEntity`, `BookingEntity`, etc.) son clases convencionales en la capa de infraestructura y no se exponen nunca fuera de ella.

5.2 Modelo de dominio

Las entidades principales del dominio son:

User: Usuario del sistema. Campos: `userId` (UUID), `firstName`, `lastName`, `email`, `passwordHash` (BCrypt), `phoneNumber`, `centerRoles`, `audit`.

Center: Centro deportivo. Campos: `centerId` (UUID), `name`, `address`, `city`, `phoneNumber`, `email`, `manager`, `audit`.

Field: Pista de un centro. Campos: `fieldId` (UUID), `name`, `type`, `pricePerHour` (BigDecimal), `isAvailable` (Boolean), `center`, `audit`.

Booking: Reserva. Campos: `bookingId` (BIGINT — excepción histórica respecto al uso de UUID en el resto de entidades), `user`, `field`, `startTime/endTime` (LocalDateTime), `totalPrice`, `status` (PENDING, CONFIRMED, CANCELLED, COMPLETED), `bookedAt`, `audit`.

CenterRole: Rol contextual de un usuario en un centro concreto. Permite que un mismo usuario sea ADMIN en un centro y USER en otro.

Tournament: Torneo. Campos: `tournamentId` (UUID), `centerId`, `name`, `format` (ROUND_ROBIN, ELIMINATION o GROUPS_AND_ELIMINATION), `status` (ver figura 4.1), `maxPairs`, `startDate`, `endDate`, `audit`.

TournamentPair: Pareja inscrita en un torneo. Estado: PENDING, que evoluciona a CONFIRMED o REJECTED. Incluye `teamName` opcional.

Match: Partido generado para un emparejamiento. Estado: SCHEDULED, IN_PROGRESS, COMPLETED, CANCELLED. Contiene `MatchResult` con marcador y ganador.

Auditable: Record base de auditoría incluido en todas las entidades: `createdAt`, `createdBy` (UUID), `modifiedAt`, `modifiedBy`, `deletedAt`, `deletedBy` (soft delete).

5.3 Diseño de la API — Contract-First

El enfoque *contract-first* implica que la especificación OpenAPI se escribe antes que cualquier código de implementación. El fichero `docs/openapi/index.yaml` es la fuente de verdad del contrato de la API.

El proceso de desarrollo de un nuevo endpoint sigue estos pasos:

1. Se añade el endpoint al fichero `index.yaml` con sus parámetros, esquemas de request y response, y códigos de estado.
2. Se ejecuta `./mvnw generate-sources`, que invoca el plugin *openapi-generator-maven-plugin 7.6.0* y genera las interfaces de controlador y los DTOs (como records Java) en `target/generated-sources`.
3. El controlador implementa la interfaz generada. Si la firma del método no coincide con la especificación, el compilador lo detecta.
4. En el frontend, `pnpm generate-api` (con el backend en :8080) regenera los hooks React Query mediante Orval.

5.3.1 Convenciones de la especificación

- Todos los endpoints siguen el prefijo `/api/v1/`.
- Los identificadores de recursos son UUIDs (salvo `bookingId`, que es BIGINT por razones históricas).

- Los errores siguen el esquema `ProblemDetail` de RFC 9457.
- Los *timestamps* se representan en formato ISO 8601 con zona horaria UTC.
- La paginación sigue el esquema de Spring Data con `page`, `size` y `sort`, envuelto en `PageResult<T>`.

5.3.2 Endpoints de la API

La tabla 5.1 recoge los 40 endpoints expuestos por la API, organizados por controlador.

5.4 Esquema de base de datos

La base de datos es PostgreSQL 16 [?]. Las migraciones se gestionan con Flyway y se nombran siguiendo el convenio `V{version}__{descripcion}.sql`.

Las tablas principales son:

- `users`: datos de usuario y credenciales (BCrypt).
- `center_roles`: relación usuario-centro-rol con restricción `UNIQUE(user_id, center_id)`.
- `centers`: centros deportivos con soft delete.
- `fields`: pistas de cada centro con precio y flag `is_available`.
- `bookings`: reservas con clave primaria `BIGSERIAL` y control de solapamiento mediante índice parcial (migración V4).
- `booking_participants`: participantes adicionales de una reserva.
- `tournaments`: torneos con `CHECK` sobre `status` (6 valores) y `format` (3 valores).
- `tournament_pairs`: parejas inscritas con `team_name` (añadido en migración V8).
- `tournament_matches`: partidos con resultado embebido.

Todas las tablas incluyen columnas de auditoría: `created_at`, `created_by` (UUID), `modified_at`, `modified_by`, `deleted_at`, `deleted_by`.

5.5 Diseño de seguridad

La autenticación se basa en JWT firmados con HMAC-SHA256 (HS256). En el lado del servidor, la verificación se implementa mediante el módulo `spring-boot-starter-oauth2-resource-server` de Spring Security, que configura un `NimbusJwtDecoder` con la clave secreta local. Esto permite aprovechar toda la infraestructura de integración JWT de Spring (decodificación, validación de claims, inyección en el contexto de seguridad) sin depender de ningún proveedor de identidad externo.

El flujo de autenticación es el siguiente:

1. El cliente envía `POST /api/v1/auth/login` con correo y contraseña.
2. El servidor verifica las credenciales y genera un *access token* (15 minutos de validez) y un *refresh token* (7 días).
3. El *access token* se devuelve en el cuerpo de la respuesta; el *refresh token* se establece como cookie `httpOnly` y `SameSite=Strict`.
4. Las peticiones protegidas incluyen el *access token* en el header `Authorization: Bearer {token}`.
5. Cuando el *access token* expira, el cliente llama a `POST /api/v1/auth/refresh`; el servidor lee la cookie y emite un nuevo par de tokens.

5.5.1 Evaluadores de autorización personalizados

Más allá de la autenticación, PadelCenter implementa autorización granular mediante evaluadores custom registrados como beans de Spring:

@bookingOwnerEvaluator: Verifica que el usuario autenticado es el propietario de la reserva, o que tiene rol `MANAGER` o `ADMIN` en el centro de la pista reservada.

@centerRoleEvaluator: Verifica que el usuario tiene el rol mínimo requerido en el centro sobre el que opera. Permite operaciones diferenciadas según si el actor es `USER`, `MANAGER` o `ADMIN` en ese centro concreto.

@tournamentRoleEvaluator: Verifica permisos específicos sobre torneos: inscripción (cualquier `USER`), confirmación de parejas y reporte de resultados (`MANAGER` del centro), gestión del ciclo de vida (`ADMIN`).

Estos evaluadores se invocan mediante `@PreAuthorize` directamente en los métodos de los casos de uso, manteniendo la lógica de autorización cerca de la lógica de negocio sin contaminar los controladores.

Método	Ruta	Descripción	Rol mínimo
POST	/api/v1/auth/login	Iniciar sesión	Anónimo
POST	/api/v1/auth/refresh	Renovar token	Anónimo
POST	/api/v1/auth/logout	Cerrar sesión	Anónimo
GET	/api/v1/me	Usuario actual	USER
GET	/api/v1/me/bookings	Mis reservas	USER
GET	/api/v1/me/tournaments	Mis torneos	USER
GET	/api/v1/me/matches	Mis partidos	USER
POST	/api/v1/users	Registrarse	Anónimo
GET	/api/v1/users	Listar usuarios	ADMIN
GET	/api/v1/users/{id}	Ver usuario	ADMIN
PUT	/api/v1/users/{id}	Actualizar usuario	Propietario/ADMIN
PUT	/api/v1/users/{id}/roles	Cambiar roles	ADMIN
PUT	/api/v1/users/{id}/password	Cambiar contraseña	Propietario
DELETE	/api/v1/users/{id}	Eliminar usuario	ADMIN
GET	/api/v1/users/{id}/bookings	Reservas de usuario	ADMIN
POST	/api/v1/centers	Crear centro	ADMIN
GET	/api/v1/centers	Listar centros	USER
GET	/api/v1/centers/{id}	Ver centro	USER
DELETE	/api/v1/centers/{id}	Eliminar centro	ADMIN
POST	/api/v1/centers/{id}/fields	Crear pista	ADMIN
GET	/api/v1/centers/{id}/fields	Listar pistas del centro	USER
GET	/api/v1/centers/{id}/bookings	Reservas del centro	MANAGER
GET	/api/v1/centers/{id}/tournaments	Torneos del centro	USER
GET	/api/v1/fields	Listar pistas	USER
GET	/api/v1/fields/{id}	Ver pista	USER
GET	/api/v1/fields/{id}/availability	Disponibilidad por fecha	USER
PUT	/api/v1/fields/{id}/availability	Activar/desactivar pista	MANAGER
DELETE	/api/v1/fields/{id}	Eliminar pista	ADMIN
POST	/api/v1/bookings	Crear reserva	USER
GET	/api/v1/bookings/{id}	Ver reserva	Propietario/MANAGER
PUT	/api/v1/bookings/{id}	Actualizar reserva	Propietario
POST	/api/v1/bookings/{id}/cancel	Cancelar reserva	Propietario/MANAGER
POST	/api/v1/tournaments	Crear torneo	ADMIN
GET	/api/v1/tournaments/{id}	Ver torneo	USER
DELETE	/api/v1/tournaments/{id}	Eliminar torneo	ADMIN
POST	/api/v1/tournaments/{id}/pairs	Inscribirse como pareja	USER
GET	/api/v1/tournaments/{id}/pairs	Ver parejas	USER
POST	/api/v1/tournaments/{id}/pairs/{pId}/confirm	Confirmar pareja	MANAGER
POST	/api/v1/tournaments/{id}/registration/open	Abrir inscripción	ADMIN
POST	/api/v1/tournaments/{id}/registration/close	Cerrar e generar cuadro	ADMIN
GET	/api/v1/tournaments/{id}/matches	Ver partidos	USER
GET	/api/v1/tournaments/{id}/standings	Ver clasificación	USER
POST	/api/v1/matches/{id}/result	Reportar resultado	MANAGER

Tabla 5.1: Endpoints de la API REST de PadelCenter.

Implementación

LA implementación de PadelCenter se divide en dos proyectos independientes que se comunican a través del contrato OpenAPI: el backend en Java 21 con Spring Boot 3.4.4 y el frontend en TypeScript con Next.js 14. En este capítulo se describen las decisiones de implementación más relevantes de cada uno y el flujo de integración end-to-end.

6.1 Backend

6.1.1 Estructura del proyecto

El proyecto backend organiza el código siguiendo la estructura de la arquitectura hexagonal, con el paquete raíz `com.bookings.padelcenter`:

- `domain.model`: records inmutables de dominio (User, Center, Field, Booking, Tournament, TournamentPair, Match, CenterRole, Auditable...). Sin dependencias externas.
- `domain.port`: interfaces de puertos de entrada y salida.
- `domain.exception`: excepciones de dominio tipadas (ResourceNotFoundException, BookingAlreadyCancelledException, EmailAlreadyExistsException, UnauthorizedException...).
- `application.create/read/update/delete`: 42 casos de uso organizados por naturaleza de la operación. Cada uno implementa `CommandUseCase` o `QueryUseCase`.
- `infrastructure.inbound.web`: controladores HTTP que implementan las interfaces generadas por OpenAPI.
- `infrastructure.inbound.mapper`: mappers `API↔dominio`.

- `infrastructure.outbound.db.entity`: entidades JPA.
- `infrastructure.outbound.db.mapper`: mappers dominio↔entidad JPA.
- `infrastructure.outbound.security`: JWT, BCrypt, evaluadores de autorización.
- `infrastructure.config`: configuración de Spring Security y logging.

6.1.2 Generación de código OpenAPI

El plugin `openapi-generator-maven-plugin` [?] (v7.6.0) se configura en el `pom.xml` para generar en fase `generate-sources`:

- **Interfaces de controlador**: anotadas con `@RequestMapping`, que los controladores implementan. Si la firma no coincide con el contrato, el compilador lo detecta.
- **DTOs como records Java**: inmutables, con anotaciones Bean Validation derivadas del esquema OpenAPI (`@NotNull`, `@Size`, etc.). Exclusivos de la capa web; nunca se exponen al dominio.

6.1.3 Patrón de mapeo en dos capas

El proyecto implementa el mapeo entre capas mediante dos familias de clases `@Component` manuales, sin dependencias de generación de código:

`infrastructure.inbound.mapper`: Mappers **API↔dominio**. Transforman los records DTO generados por OpenAPI en objetos de dominio (comandos y queries) antes de invocar el caso de uso, y transforman el resultado del caso de uso en el DTO de respuesta. Por ejemplo, `BookingApiMapper` convierte `CreateBookingRequest` en `CreateBookingUseCase.Command` y `Booking` en `BookingResponse`.

`infrastructure.outbound.db.mapper`: Mappers **dominio↔entidad JPA**. Transforman los records de dominio en entidades anotadas con `@Entity` y viceversa, gestionando la conversión de tipos (UUID, `BigDecimal`, enumerados) y la hidratación de objetos relacionados. Por ejemplo, `BookingPersistenceMapper` transforma entre `Booking` (record inmutable) y `BookingEntity` (entidad JPA mutable).

Esta separación garantiza que ni los DTOs de la API ni las entidades JPA contaminan el dominio o los casos de uso.

6.1.4 Flyway y migraciones de base de datos

El esquema se gestiona con Flyway [?]. Las migraciones viven en `src/main/resources/db/migration/`:

- `V1__init_schema.sql`: tablas `users`, `centers`, `fields`, `bookings`, `booking_participants` y `center_roles` con columnas de auditoría.
- `V3__add_missing_indexes.sql`: índices de optimización.
- `V4__add_booking_overlap_index.sql`: índice parcial para detección eficiente de solapamiento de reservas a nivel de base de datos.
- `V5__create_tournament_tables.sql`: crea las tablas de torneos (`tournaments`, `tournament_pairs` y `tournament_matches`).
- `V8__add_team_name_to_pairs.sql`: campo `team_name` opcional en `tournament_pairs`.

Además de las migraciones de esquema, el perfil `local` carga una ubicación adicional, `src/main/resources/db/seed/`, con datos de prueba para desarrollo manual:

- `V9__seed_curated_data.sql`: conjunto de datos curado a mano (un usuario ADMIN, 6 centros, 24 pistas, un torneo con 8 parejas confirmadas, etc.) con identificadores UUID fijos, de forma que sigan siendo válidos tras un `docker compose down -v` que recree la base de datos desde cero.
- `V10__seed_bulk_data.sql`: volumen adicional (180 usuarios y ~1000 reservas) generado con `generate_series` de PostgreSQL, para probar paginación y filtros con datos realistas.

Esta ubicación solo se añade a `spring.flyway.locations` cuando el perfil activo es `local` (ver `application-local.yaml`), por lo que nunca se aplica en el perfil `test` ni en un despliegue real.

6.1.5 Virtual threads (Project Loom)

El servidor usa virtual threads de Java 21 [?], activados con una única propiedad:

```
1 spring.threads.virtual.enabled=true
```

Los virtual threads permiten un modelo de concurrencia de alta densidad: como son baratos de crear y bloquear, el servidor puede manejar miles de peticiones simultáneas sin el overhead de un pool de hilos de plataforma.

6.1.6 Trazabilidad con MDC y logging estructurado

Todas las peticiones HTTP pasan por un filtro `MdcFilter` que inyecta en el MDC (*Mapped Diagnostic Context*) el identificador de usuario y un ID de correlación de petición. Los logs se emiten en formato JSON mediante *logstash-logback-encoder*, lo que facilita la ingesta en sistemas de observabilidad como Elasticsearch o Loki.

6.1.7 Convenciones de los casos de uso

Cada caso de uso implementa `CommandUseCase<C, R>` o `QueryUseCase<Q, R>`, donde el tipo genérico de entrada es siempre un record interno de la propia clase. Esto agrupa comando y caso de uso en un único fichero y hace explícito el contrato de cada operación. Las excepciones de dominio se propagan hasta el `GlobalExceptionHandler` (`@ControllerAdvice`), que las traduce a respuestas `ProblemDetail` según RFC 9457.

6.2 Frontend

6.2.1 Estructura del proyecto

El frontend usa el App Router de Next.js 14 [?]. Las rutas se organizan en grupos entre paréntesis para compartir layouts sin añadir segmentos a la URL:

- `app/(auth)/`: rutas autenticadas con layout de aplicación (header, navegación). Incluye: `dashboard`, `centers`, `centers/[centerId]`, `bookings`, `tournaments`, `tournaments/[tournamentId]`, `matches` y `profile`.
- `app/(admin)/admin/`: rutas exclusivas de administrador. Incluye: gestión de `centers`, `centers/[centerId]`, `fields`, `bookings` y `tournaments`.
- Rutas públicas: `/login`, `/register`, `/auth/callback`.

6.2.2 Librería de componentes: shadcn/ui y Radix UI

La interfaz se construye sobre **shadcn/ui** [?], una colección de componentes copiables basada en primitivos accesibles de **Radix UI** [?] y estilizada con Tailwind CSS [?]. Esta elección aporta:

- Componentes accesibles (ARIA) sin configuración adicional (Dialog, Popover, DropdownMenu, Select, Tabs, Toast...).

- Control total sobre el código: los componentes se copian al repositorio y son modificables, a diferencia de librerías opacas de terceros.
- Integración nativa con Tailwind CSS y `class-variance-authority` para variantes tipadas de estilos.

6.2.3 Generación de hooks con Orval

Orval [?] lee la especificación OpenAPI del backend corriendo en :8080 y genera en `src/lib/api/generated/`, organizado por tag (auth, bookings, centers, fields, matches, tournaments, users):

- Un hook React Query [?] (`useQuery`) por operación GET.
- Una función de mutación (`useMutation`) por operación POST/PUT/DELETE.
- Los tipos TypeScript correspondientes a todos los esquemas del OpenAPI.

El cliente HTTP subyacente es Axios [?] con un mutator personalizado que gestiona el refresco automático del token cuando el servidor responde con HTTP 401.

6.2.4 Gestión del estado de autenticación con Zustand

El `access token` JWT se almacena en memoria en un store Zustand [?] (`src/lib/auth/store.ts`), nunca en `localStorage` ni en `sessionStorage`, lo que elimina el riesgo de robo mediante XSS. Los datos del usuario se persisten en `localStorage` mediante el middleware `persist` de Zustand para sobrevivir a recargas de página. Al recargar, el interceptor de Axios intenta el refresco del token mediante la cookie `httpOnly` antes de redirigir al login.

6.2.5 Protección de rutas

El middleware de Next.js (`middleware.ts`) actúa como guardián de rutas: verifica la autenticación para las rutas bajo `/ (auth) /` y además comprueba el rol ADMIN para las rutas bajo `/ (admin) /`, redirigiendo a `/dashboard` si el rol es insuficiente.

6.3 Flujo end-to-end: creación de una reserva

A continuación se describe el flujo completo desde el clic del usuario hasta la respuesta en pantalla:

1. El usuario selecciona pista, fecha y franja horaria y pulsa *Reservar*.

2. El hook `useCreateBooking` (generado por Orval) envía `POST /api/v1/bookings` con el *access token* JWT en el header `Authorization: Bearer`.
3. El filtro de Spring Security decodifica el token con `NimbusJwtDecoder` y establece el contexto de seguridad.
4. `BookingController` recibe la petición, la valida con `Bean Validation`, usa `BookingApiMapper` para transformar el DTO en un `CreateBookingUseCase.Command` y ejecuta el caso de uso.
5. `CreateBookingUseCase` invoca el método `existsConflict()` de `BookingRepository`, que consulta el índice de solapamiento de la base de datos.
6. Si no hay conflicto, persiste la reserva con estado `PENDING`, calcula el precio total y devuelve el record `Booking`.
7. El controlador usa `BookingApiMapper` para transformar el record en `BookingResponse` y responde con `HTTP 201`.
8. `React Query` invalida la caché de `useGetMyBookings` y el listado se actualiza automáticamente.

6.4 Entorno local con Docker Compose

El fichero `tools/docker/docker-compose.yaml` define los servicios necesarios para el desarrollo local:

- **postgres:** PostgreSQL 16, expuesto en el puerto 5432. Flyway aplica las migraciones automáticamente al arrancar el backend.
- **padelcenter:** backend Spring Boot con perfil de desarrollo.

El frontend se ejecuta fuera de Docker con `pnpm dev` para aprovechar el *hot reload* de Next.js. La variable de entorno `NEXT_PUBLIC_API_URL=http://localhost:8080` conecta ambos proyectos.

6.4.1 Colección Postman

`tools/postman/` incluye una colección con las 48 operaciones de la API, organizada por recurso, y un *environment* (`PadelCenter - Local`) con las credenciales y los identificadores fijos de los datos curados de V9. Cada petición que crea un recurso captura su

identificador en una variable del entorno mediante un script de test, de forma que la colección puede recorrerse de principio a fin sin intervención manual. La validación sistemática de las 48 peticiones con `newman`, la CLI de Postman, contra el entorno local fue precisamente el método que permitió detectar varios de los defectos corregidos durante el desarrollo (véase la sección 7.5).

Capítulo 7

Pruebas

LA estrategia de pruebas de PadelCenter sigue la pirámide de pruebas clásica: una base amplia de tests unitarios rápidos, una capa intermedia de tests de integración que verifican el comportamiento con recursos reales, y tests de arquitectura que garantizan el cumplimiento de las reglas de capas. Este enfoque maximiza la confianza en el sistema minimizando el tiempo total de ejecución del conjunto de pruebas. La escritura de tests unitarios antes o inmediatamente después de cada caso de uso, descrita por Beck como práctica central del desarrollo dirigido por pruebas [?], ha guiado además el diseño de las interfaces de los puertos del dominio: un puerto difícil de testear de forma aislada es, en la mayoría de los casos, síntoma de un diseño con demasiadas responsabilidades.

7.1 Estrategia de pruebas

Tests unitarios: Verifican el comportamiento de un caso de uso o un componente de dominio de forma aislada, con las dependencias sustituidas por dobles de prueba (Mockito). Son muy rápidos y no requieren infraestructura externa.

Tests de integración: Verifican el comportamiento del sistema de extremo a extremo a nivel HTTP, usando una base de datos PostgreSQL real gestionada por Testcontainers. Detectan errores que los tests unitarios no pueden encontrar: consultas JPA incorrectas, problemas de transaccionalidad, validaciones de esquema, etc.

Tests de arquitectura: Verifican que la estructura del código cumple las reglas de la arquitectura hexagonal (ninguna clase del dominio importa clases de Spring, los casos de uso no dependen de JPA, etc.). Se ejecutan en cada build y fallan si alguien introduce una dependencia prohibida.

7.2 Tests unitarios con Mockito

Los tests unitarios usan Mockito [?] y siguen el patrón *given/when/then* y la convención de nombres `should{Behavior}When{Context}()`. Por ejemplo:

```
1 @Test
2 void shouldThrowConflictExceptionWhenBookingOverlaps() {
3     // given
4     given(bookingRepository.existsConflict(FIELD_ID, START, END))
5         .willReturn(true);
6
7     // when / then
8     assertThatThrownBy(() -> useCase.execute(command))
9         .isInstanceOf(BookingConflictException.class);
10 }
```

Cada clase de caso de uso tiene su correspondiente clase de test. Los dobles de prueba se crean con `@Mock` y la clase bajo test con `@InjectMocks` en combinación con `MockitoExtension`.

Los objetos de dominio que contienen lógica de negocio (por ejemplo, la validación de solapamiento de horarios en `Booking`) también tienen tests unitarios propios, sin mocks, que verifican el comportamiento con entradas y salidas concretas.

7.3 Tests de integración con Testcontainers

Los tests de integración usan `@SpringBootTest(webEnvironment = RANDOM_PORT)` y un contenedor PostgreSQL gestionado por Testcontainers [?]:

```
@Container
static PostgreSQLContainer<?> postgres =
    new PostgreSQLContainer<>("postgres:16")
        .withDatabaseName("padelcenter_test");
```

El contenedor se levanta una sola vez por clase de test (`@TestcontainersExtension`) para minimizar el overhead. Flyway aplica las migraciones automáticamente al arrancar el contexto Spring, garantizando que el esquema de test siempre coincide con el de producción.

Las peticiones HTTP se realizan con `TestRestTemplate` o con el cliente Orval generado en modo test, cubriendo el ciclo completo:

1. Autenticación previa para obtener un JWT válido.
2. Llamada al endpoint bajo prueba con el token en el header.

3. Verificación del código de estado HTTP y del cuerpo de la respuesta.
4. Verificación directa en base de datos para casos críticos de escritura.

7.4 Tests de arquitectura con ArchUnit

ArchUnit [?] permite expresar las reglas de arquitectura como código Java y verificarlas en tiempo de compilación. PadelCenter define las siguientes reglas:

- Las clases del paquete `domain` no deben depender de ninguna clase de los paquetes `org.springframework` ni `jakarta.persistence`.
- Las clases del paquete `application` solo pueden depender de `domain`, no de `infrastructure`.
- Todas las clases de casos de uso deben tener exactamente un método público llamado `execute`.
- Los nombres de las clases de repositorio JPA deben terminar en `JpaRepository`.

Si alguna de estas reglas se viola, el test falla con un mensaje descriptivo que indica exactamente qué clase incumple la regla y por qué.

7.5 Resumen de cobertura

La tabla 7.1 muestra el número de tests por tipo y la cobertura global del backend, medida con JaCoCo [?] a partir de la fusión de los datos de ejecución de Surefire (unitarios) y Failsafe (integración) — `mvn clean verify`. El código generado a partir del contrato OpenAPI (DTOs e interfaces de controlador) se excluye del cálculo, ya que no es código escrito a mano.

Tipo	Tests	Capas cubiertas	Cobertura (líneas)
Unitarios (Mockito)	193	<code>domain</code> , <code>application</code>	91,4%
Integración (Testcontainers)	52	<code>infrastructure</code> + end-to-end	81,1%
Arquitectura (ArchUnit)	5	Reglas de capas	N/A
Total	250	—	85,3%

Tabla 7.1: Resumen de tests y cobertura del backend (medida con JaCoCo, perfil `mvn clean verify`).

La cobertura del 91,4% en `domain` y `application` refleja que las capas más críticas del sistema están ampliamente verificadas mediante tests unitarios rápidos. La capa de infraestructura alcanza el 81,1% gracias a ocho clases `*ControllerIT`, una por cada controlador

REST (UserControllerIT, CenterControllerIT, FieldControllerIT, BookingControllerIT, TournamentControllerIT, MatchControllerIT, AuthControllerIT) más SecurityConfigPublicEndpointsIT para la configuración de seguridad. El hueco restante se concentra en GlobalExceptionHandler (50,9%): cada @ExceptionHandler solo se cubre si algún test llega a provocar esa excepción concreta, y varias —por ejemplo BookingOverlapException o PairAlreadyRegisteredException— todavía no tienen un test de integración dedicado. Cerrar esa brecha por completo queda como trabajo futuro en la sección 8.

Conclusiones

EL desarrollo de PadelCenter ha permitido poner en práctica, en un proyecto real de alcance completo, las competencias adquiridas durante el Grado en Ingeniería Informática, mención Ingeniería del Software. En este capítulo se evalúan los resultados obtenidos, se analizan las dificultades encontradas y se proponen líneas de trabajo futuro.

8.1 Objetivos alcanzados

Todos los objetivos planteados en la sección 1.2 han sido satisfactoriamente alcanzados:

- Se ha implementado un sistema completo de autenticación JWT con refresh token via cookie *httpOnly*, cobertura de roles y protección de rutas tanto en el backend como en el frontend.
- El módulo de gestión de centros, pistas y reservas funciona correctamente, incluyendo el control de conflictos temporales y la diferenciación de visibilidad según el rol del usuario.
- El módulo de torneos permite la inscripción de jugadores, la generación automática de emparejamientos y el registro de resultados.
- La arquitectura hexagonal se ha aplicado de forma rigurosa y se verifica automáticamente en cada build mediante ArchUnit.
- El enfoque *contract-first* con OpenAPI es efectivo: toda la API está documentada en el YAML y tanto los interfaces del backend como los hooks del frontend se generan automáticamente a partir de él.
- El entorno de desarrollo es completamente reproducible con Docker Compose.

8.2 Dificultades encontradas

Integración Orval con App Router de Next.js La generación de clientes HTTP con Orval no es trivial cuando se combina con el App Router de Next.js 14, que distingue entre *Server Components* y *Client Components*. Los hooks React Query solo pueden usarse en componentes de cliente, lo que requiere una estrategia de prefetch en el servidor y uso de `use client` en los componentes que consumen los hooks.

Control transaccional en tests de integración Los tests de integración con Testcontainers y Spring requieren un cuidado especial en el aislamiento de datos entre tests. El uso de `@Transactional` en los tests puede enmascarar problemas reales con las transacciones de la aplicación, por lo que se optó por truncar las tablas entre tests de forma explícita.

Emparejamientos con número de parejas no potencia de 2 El formato ELIMINATION necesita, en cada ronda, que el número de parejas se pueda dividir entre dos hasta llegar a la final. Con un número de parejas que no es potencia de 2 hacen falta *byes* (pases automáticos a la siguiente ronda para las parejas sobrantes), lo que complica tanto el cálculo de quién recibe el *bye* —debe repartirse de forma justa, no siempre al mismo cabeza de serie— como la generación de los partidos *placeholder* de rondas posteriores. Tras valorar esa complejidad frente al beneficio real para un sistema de gestión de un club de pádel, se optó deliberadamente por **no** implementarla: `CloseRegistrationAndGenerateBracketUseCase` exige que el número de parejas confirmadas sea potencia de 2 para ELIMINATION y rechaza el cierre de inscripción en caso contrario (sección 4.2). Es una decisión consciente de simplicidad sobre generalidad, no una limitación accidental: soportar byes queda como posible ampliación futura si la casuística real de los clubes lo demanda.

8.3 Relación con las competencias de la titulación

El desarrollo de PadelCenter ha permitido adquirir y consolidar las siguientes competencias del GEL, mención Ingeniería del Software:

CE1 — Accesibilidad, usabilidad y seguridad: El diseño de la API REST, la implementación de la autenticación JWT y la protección de rutas demuestran la aplicación de esta competencia.

CE2 — Plataformas hardware y software: La evaluación comparativa de frameworks (Spring Boot, Quarkus, Next.js) y la justificación de las elecciones tecnológicas refleja esta competencia.

CE3 — Calidad e innovación tecnológica: La estrategia de pruebas (unitarios, integración, arquitectura) y la verificación automática de las reglas de arquitectura son manifestaciones de esta competencia.

CE4 — Servicios y sistemas software: El ciclo completo desde el análisis de requisitos hasta la implementación y validación mediante tests demuestra esta competencia.

CE5 (mención) — Especificación, diseño e implementación: La aplicación de arquitectura hexagonal, el enfoque *contract-first* y el uso de herramientas modernas de generación de código son ejemplos concretos de esta competencia aplicada.

8.4 Consideraciones éticas, legales y de sostenibilidad

8.4.1 Protección de datos personales

PadelCenter trata datos de carácter personal: nombre, correo electrónico, teléfono y contraseña (esta última nunca en claro, sino como *hash* BCrypt). El tratamiento de estos datos debe ajustarse al Reglamento General de Protección de Datos (RGPD) y a la Ley Orgánica 3/2018 (LOPDGDD). El diseño actual cumple algunos principios básicos —minimización (solo se solicitan los campos estrictamente necesarios para operar el sistema) y seguridad (*hashing* de contraseñas, cifrado en tránsito mediante HTTPS en despliegue, *token* de refresco en cookie *httpOnly*)— pero también revela una tensión no resuelta: el uso de *soft delete* (sección 5.4) mantiene los datos del usuario en base de datos tras su eliminación lógica, lo que entra en conflicto con el derecho al olvido si no se complementa con un proceso de purga definitiva tras el plazo legal de conservación. Una versión productiva del sistema debería incorporar dicho proceso de purga y un registro de actividades de tratamiento.

8.4.2 Impacto social y económico

El modelo de negocio de las plataformas dominantes del sector (Playtomic, Smashapp) se basa en una comisión por reserva, lo que penaliza proporcionalmente más a los clubes pequeños con menor volumen de reservas. PadelCenter, al ser una solución que un club puede operar de forma autónoma, elimina esa comisión variable, lo que resulta especialmente relevante para clubes de barrio o de gestión municipal con presupuestos ajustados. El efecto social esperado es una digitalización más accesible para este segmento, que de otro modo continuaría dependiendo de hojas de cálculo o llamadas telefónicas.

8.4.3 Sostenibilidad

El impacto ambiental directo del proyecto es reducido, al tratarse de software de gestión sin componente hardware específico. Cabe señalar dos decisiones con una incidencia indirecta: el uso de *virtual threads* (sección 6.1) permite atender más peticiones concurrentes con los mismos recursos de cómputo, y la sustitución de procesos manuales (llamadas, registros en papel) por una gestión digital reduce el consumo de recursos físicos asociado a la operativa diaria de un club.

8.4.4 Viabilidad económica

Una estimación del coste de desarrollo, calculada a partir del esfuerzo dedicado al proyecto (sección 3.3) y de una tarifa de mercado para un perfil de ingeniero *junior* en España (aproximadamente 20–25 €/hora en modalidad *freelance*), sitúa el coste de construir una primera versión funcional como la descrita en esta memoria en un rango de unos pocos miles de euros. Frente a esto, un modelo de negocio basado en una cuota de suscripción mensual fija por centro —en lugar de una comisión por reserva— resultaría rentable para un club con un volumen de reservas moderado en un plazo de pocos meses, y elimina la incertidumbre de ingresos variables tanto para el club como para el operador de la plataforma. Una validación rigurosa de esta hipótesis de negocio requeriría, no obstante, datos reales de adopción que quedan fuera del alcance de este TFG.

8.5 Líneas de trabajo futuro

PadelCenter es un sistema funcional y extensible, pero quedan varias líneas de trabajo que mejorarían significativamente el producto:

Autenticación OAuth2/OIDC: Integración con un proveedor de identidad como Keycloak o Auth0, permitiendo el inicio de sesión con Google u otras cuentas sociales. La arquitectura hexagonal facilita este cambio: se sustituye el adaptador JWT por un adaptador OAuth2 sin tocar el dominio.

Notificaciones push: Envío de recordatorios de reserva y notificaciones de torneos mediante WebSockets o Server-Sent Events, complementado con notificaciones push en dispositivos móviles.

Aplicación móvil: Desarrollo de una aplicación React Native que reutilice los mismos hooks generados por Orval, ya que la especificación OpenAPI también puede generar clientes para React Native.

Despliegue en cloud: Contenerización completa con Docker y despliegue en un proveedor cloud (AWS, GCP o Azure) mediante Kubernetes o un servicio PaaS. La base de datos se migraría a un servicio gestionado (RDS, Cloud SQL).

Sistema de valoraciones: Los jugadores podrían valorar las pistas y los centros tras una reserva, añadiendo un módulo de reputación al sistema.

Integración de pagos: Integración con una pasarela de pago (Stripe) para gestionar el cobro de reservas de forma automática, incluyendo reembolsos en caso de cancelación.

Cerrar la cobertura de `GlobalExceptionHandler`: Como se detalla en la sección 7.5, cada `@ExceptionHandler` solo se cubre si algún test provoca esa excepción concreta; faltan tests de integración que fuercen casos como `BookingOverlapException` o `PairAlreadyRegisteredException`. Durante el desarrollo, la validación manual de la API completa con la colección Postman (sección 6.4) detectó varios defectos —entre ellos, borrados lógicos que no se persistían realmente y una confirmación de pareja de torneo que no comprobaba a qué torneo pertenecía— que tests de integración para los controladores de torneos, partidos y autenticación (añadidos después, `TournamentControllerIT`, `MatchControllerIT` y `AuthControllerIT`) habrían detectado de forma automática mucho antes.

Apéndices

Material adicional

ESTE anexo recoge fragmentos de código e implementación que no resultan imprescindibles para seguir el cuerpo principal de la memoria, pero que ilustran con detalle dos decisiones técnicas mencionadas en los capítulos 5 y 6: la detección de solapamiento de reservas a nivel de base de datos y el algoritmo de generación de emparejamientos de torneos.

A.1 Migración V4 — Índice de solapamiento de reservas

La detección de conflictos de horario (RF-14, sección 4.2) podría implementarse comprobando en memoria todas las reservas existentes de una pista, pero esto no escala y, sobre todo, no es seguro frente a condiciones de carrera entre peticiones concurrentes. La migración `V4__add_booking_overlap_index.sql` añade un índice parcial que permite delegar esa comprobación en PostgreSQL:

```
1 CREATE INDEX idx_bookings_field_time_status
2   ON bookings (field_id, start_time, end_time)
3   WHERE status IN (1, 2);
```

El índice es **parcial**: solo cubre las reservas en estado `PENDING` (1) o `CONFIRMED` (2), que son las únicas relevantes para el cálculo de solapamiento. Las reservas `CANCELLED` y `COMPLETED` quedan fuera, lo que mantiene el índice pequeño y rápido incluso a medida que crece el histórico de reservas del sistema. La consulta de solapamiento (`BookingRepository.existsConflict`) se apoya directamente en este índice para resolver en una sola consulta SQL si existe ya una reserva activa que se solape con el rango horario solicitado.

A.2 Algoritmo de generación de cuadros de torneo

BracketGeneratorService es un servicio de dominio puro —sin dependencias de persistencia ni de Spring más allá de la anotación `@Service`— invocado por `CloseRegistrationAndGenerateBracketUseCase` al cerrar la inscripción de un torneo. Selecciona el algoritmo según el formato de competición:

```

1 public List<Match> generate(
2     Tournament tournament, List<TournamentPair> confirmedPairs)
3 {
4     return switch (tournament.format()) {
5         case ROUND_ROBIN ->
6             generateRoundRobin(tournament, confirmedPairs);
7         case ELIMINATION ->
8             generateElimination(tournament, confirmedPairs);
9         case GROUPS_AND_ELIMINATION ->
10            generateGroupsAndElimination(tournament,
11            confirmedPairs);
12    };
13 }

```

Para el formato `ROUND_ROBIN` se genera un partido por cada pareja de parejas distintas (todos contra todos):

```

1 private List<Match> generateRoundRobin(
2     Tournament t, List<TournamentPair> pairs) {
3     var matches = new ArrayList<Match>();
4     for (int i = 0; i < pairs.size(); i++) {
5         for (int j = i + 1; j < pairs.size(); j++) {
6             var a = pairs.get(i).pairId();
7             var b = pairs.get(j).pairId();
8             matches.add(buildMatch(t.tournamentId(), a, b, 1,
9             null));
10        }
11    }
12    return matches;
13 }

```

Para el formato `ELIMINATION`, las parejas se mezclan aleatoriamente y se emparejan para la primera ronda; las rondas siguientes se crean como partidos *placeholder* sin parejas asignadas (`pairAId` y `pairBId` a `null`), que se completan progresivamente a medida que se conocen los ganadores de la ronda anterior:

```

1 private List<Match> generateElimination(
2     Tournament t, List<TournamentPair> pairs) {
3     var shuffled = new ArrayList<>(pairs);

```

```

4      Collections.shuffle(shuffled);
5
6      var matches = new ArrayList<Match>();
7      int n = shuffled.size();
8
9      // Ronda 1: parejas reales
10     for (int i = 0; i < n; i += 2) {
11         var a = shuffled.get(i).pairId();
12         var b = shuffled.get(i + 1).pairId();
13         matches.add(buildMatch(t.tournamentId(), a, b, 1, null));
14     }
15
16     // Rondas siguientes: placeholders, se rellenan
17     // al completarse la ronda anterior
18     int round = 2;
19     int matchesInRound = n / 4;
20     while (matchesInRound >= 1) {
21         for (int i = 0; i < matchesInRound; i++) {
22             matches.add(
23                 buildMatch(t.tournamentId(), null, null, round,
24                 null));
25             }
26             round++;
27             matchesInRound /= 2;
28         }
29     return matches;
30 }

```

Esta estrategia de *placeholders* evita tener que regenerar la estructura completa del cuadro cuando se reporta el resultado de un partido: `CloseRegistrationAndGenerateBracketUseCase` solo necesita localizar el partido de la ronda siguiente correspondiente y rellenar el hueco vacante con el identificador del ganador.

El formato `GROUPS_AND_ELIMINATION` combina ambas estrategias: reparte las parejas en grupos de cuatro y aplica *round robin* dentro de cada grupo, seguido de una fase eliminatoria con *placeholders* para los dos primeros clasificados de cada grupo.

